Cuisine Classification from Recipe Ingredients using MultiClass Logistic Regression

Pritosh kumar - 2025JRB2048

Adarsh Pandey - 2025JRB2021

AIL7024 - Machine Learning

Guide - Prof. Tanmoy Sir

TABLE OF CONTENTS

- Introduction
- Motivation
- Problem Statement & Setup
- Key Equations / Model
- EDA
- Logistic Regression Model
- Results
- Conclusion

Problem Statement

- Given the list of ingredients in a recipe, the task is to predict its cuisine category, such as:
  • Indian, Mexican, Italian, Japanese, American etc.
  This is a multi-class classification problem.

- **Dataset Description:**

- Format: JSON
- Train samples: 29,774 recipes
- Test samples: 10,000 recipes
- Features:
  - Recipe ID
  - Ingredients list (variable length)
- Target: Cuisine label (20 classes)

Example of the data:


```
{
  "id": 24717,
  "cuisine": "indian",
  "ingredients": [
    "turmeric",
    "vegetable stock",
    "tomatoes",
    "garam masala",
    "naan",
    "red lentils",
    "red chili peppers",
    "onions",
    "spinach",
    "sweet potatoes"
  ]
}
```

Challenges in Dataset

- **Ingredients not standardized**
- Converting list of ingredients → **clean, consistent format** (lowercase, trimmed, no extra spaces).
- Customizing TfidfVectorizer to treat **each ingredient string as one feature token**:
  - Using tokenizer=lambda x: x and preprocessor=lambda x: x
  - Setting token_pattern=None to avoid default tokenization.
- Handling **sparse TF-IDF matrices** with shape ~ (29774, 4479) and ensuring code works with sparse inputs.
  - ***Note : min_df= 1 -*** **(6203 features), above one is with min_df = 2.**
- **Ingredients are multi-word expressions**

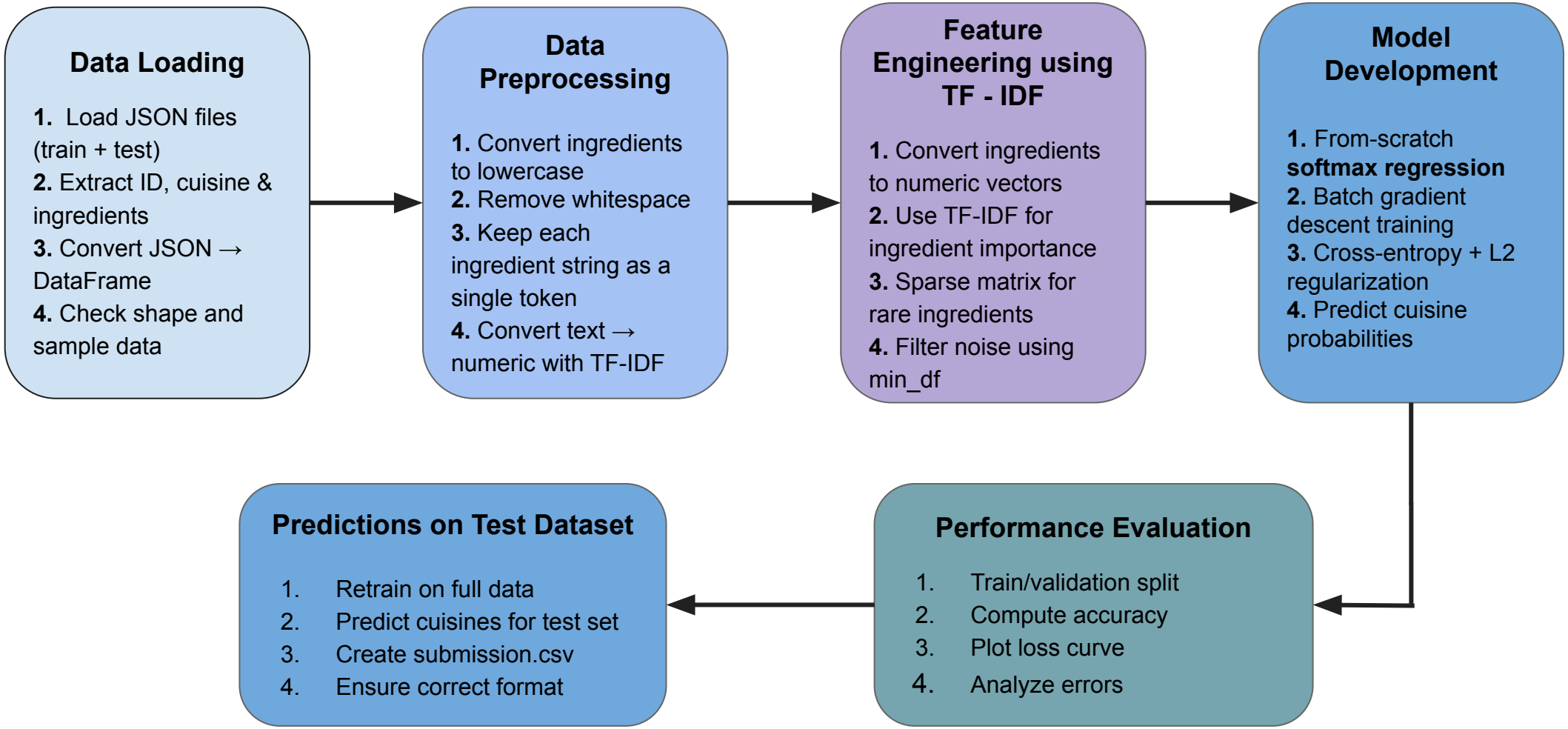
```
def clean_ingredients(ing_list):
    # lower-case and strip spaces for consistency
    return [i.lower().strip() for i in ing_list]

train["ingredients_clean"] = train["ingredients"].apply(clean_ingredients)
test["ingredients_clean"] = test["ingredients"].apply(clean_ingredients)

# 3. Create TF-IDF features
#    Each ingredient phrase is treated as one feature
vectorizer = TfidfVectorizer(
    tokenizer=lambda x: x,        # we give it a list of tokens directly
    preprocessor=lambda x: x,     # do not turn list to string
    token_pattern=None,           # because we override tokenizer
    min_df=1                      # ignore ingredients that appear in only 1 recipe
)

X = vectorizer.fit_transform(train["ingredients_clean"])
X_test_final = vectorizer.transform(test["ingredients_clean"])
y = train["cuisine"]
```

Project Workflow



Exploratory Data Analysis (EDA)

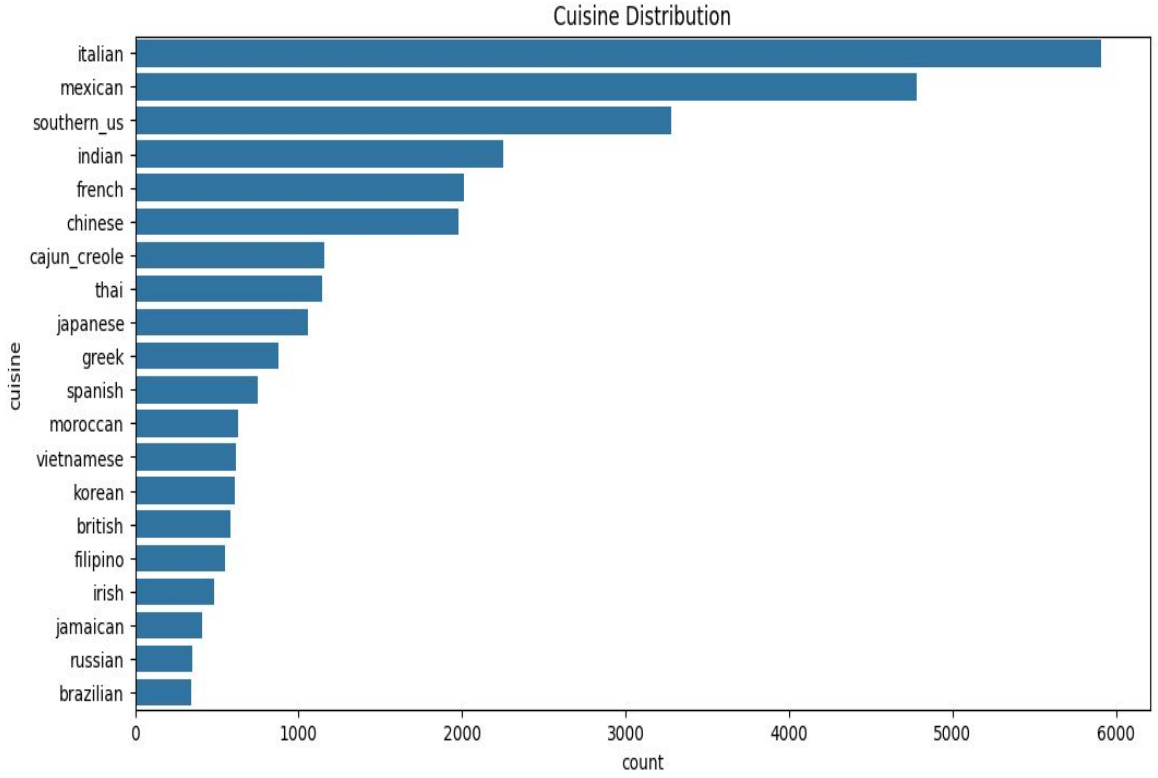


fig. 1: Cuisine Distribution (Class Imbalance)

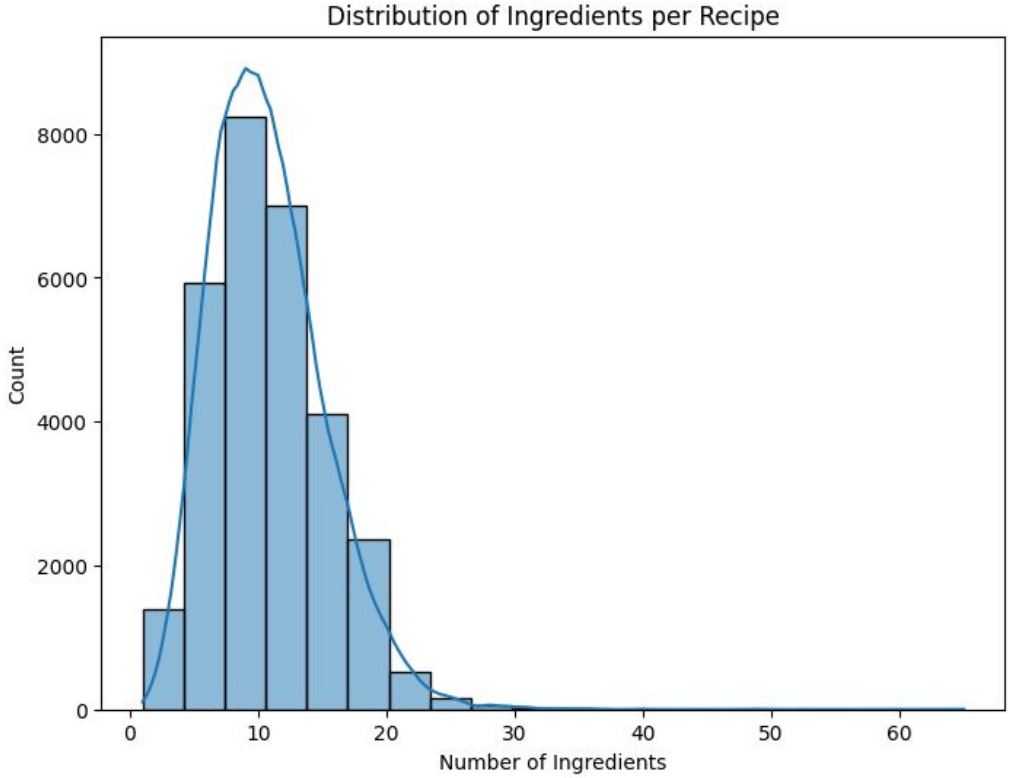


fig. 2: Ingredients Count per Recipe

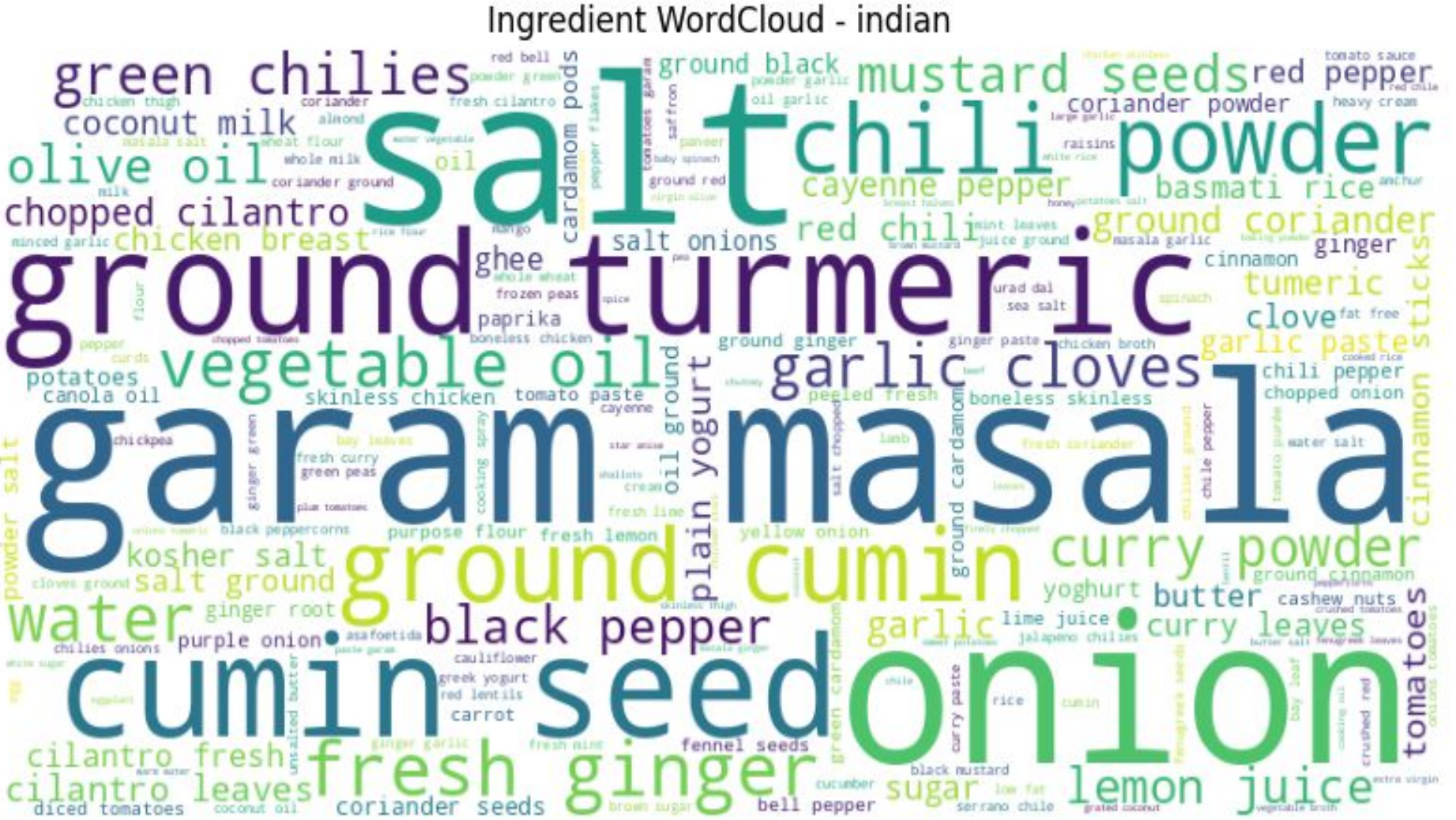
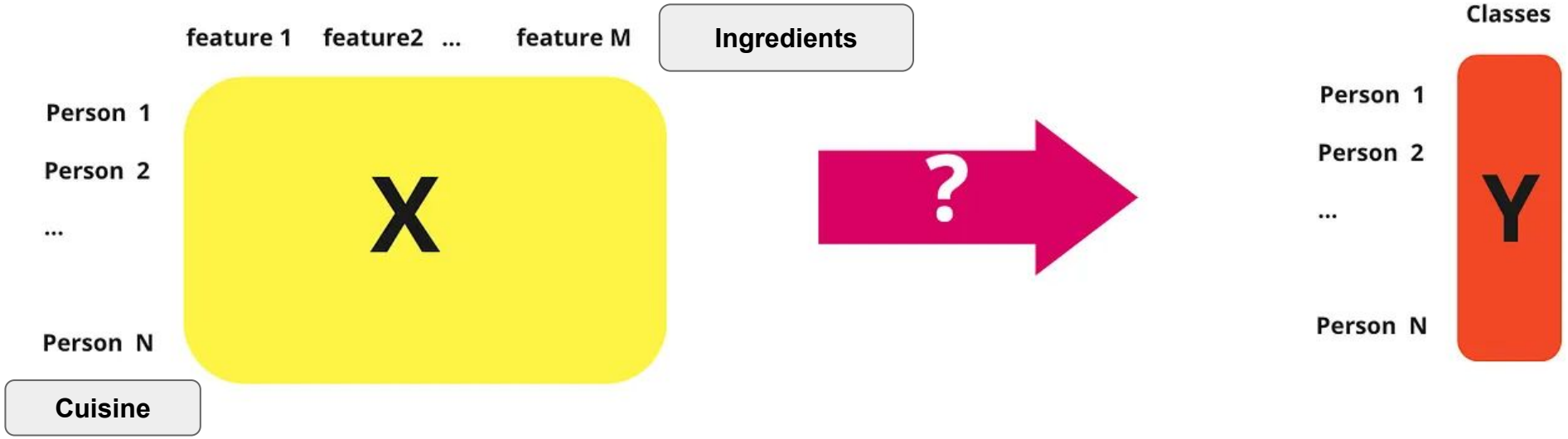


fig. 3: WordCloud showing Ingredients size based on its frequency

Model - Multiclass Logistic Regression

- Let's assume we have N people/observations, each person has M features, and they belong to C classes. We are given:
  - A matrix X is $\mathbb{R}(N \times M)$. Xij represents person i with feature j.
  - A vector Y is $\mathbb{R}(N)$. Yirepresents person i belonging to class k

- We do not know : T**he weight matrix W is $\mathbb{R}(M \times C)$.Wjk represents the weights foreature j and class k**



Multiclass logistic regression workflow

1 Matrix Multiplication

$$Z = -XW$$

- X: Feature matrix
- W: Weight matrix (initially zeros)
- Negative sign optional (does not change cononcept)
- Bias not included here for sinplicity

2 Softmax Activation

$$P_i = \text{softmax}(Z\frac{e^{Z_i}}{Z^k}$$

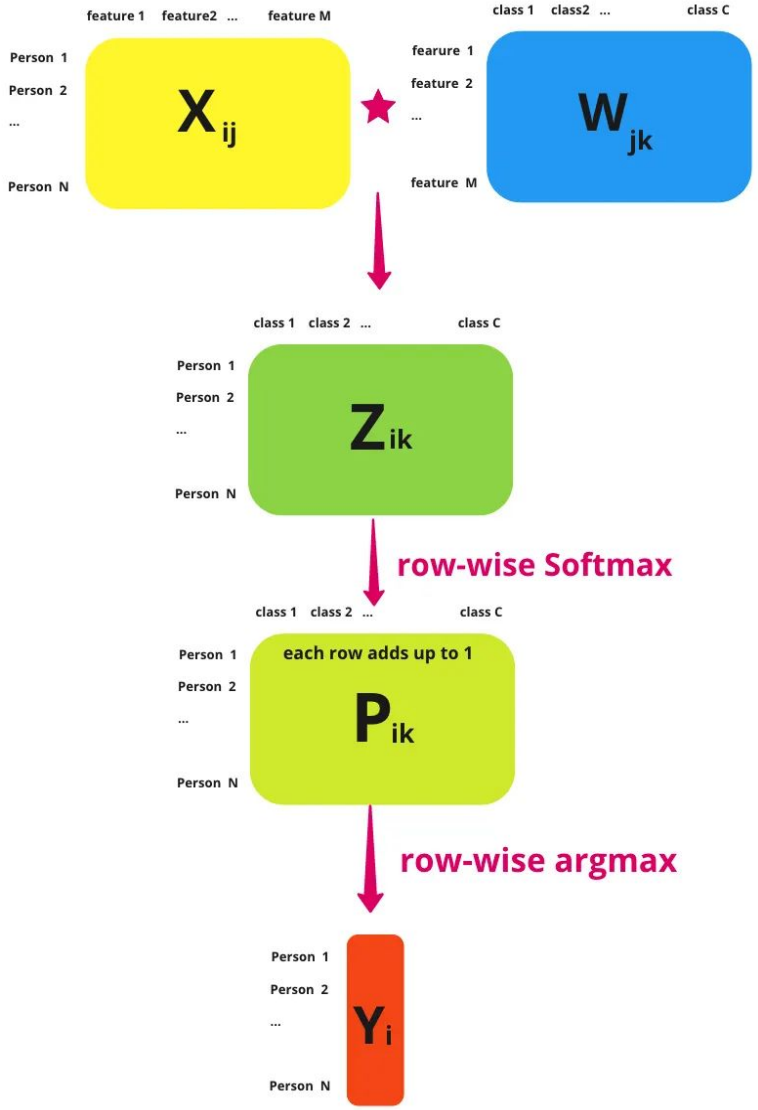
Convert raw scores into probability distribution

- Each row of P sums to 1
- Represents probability of every cuisine class

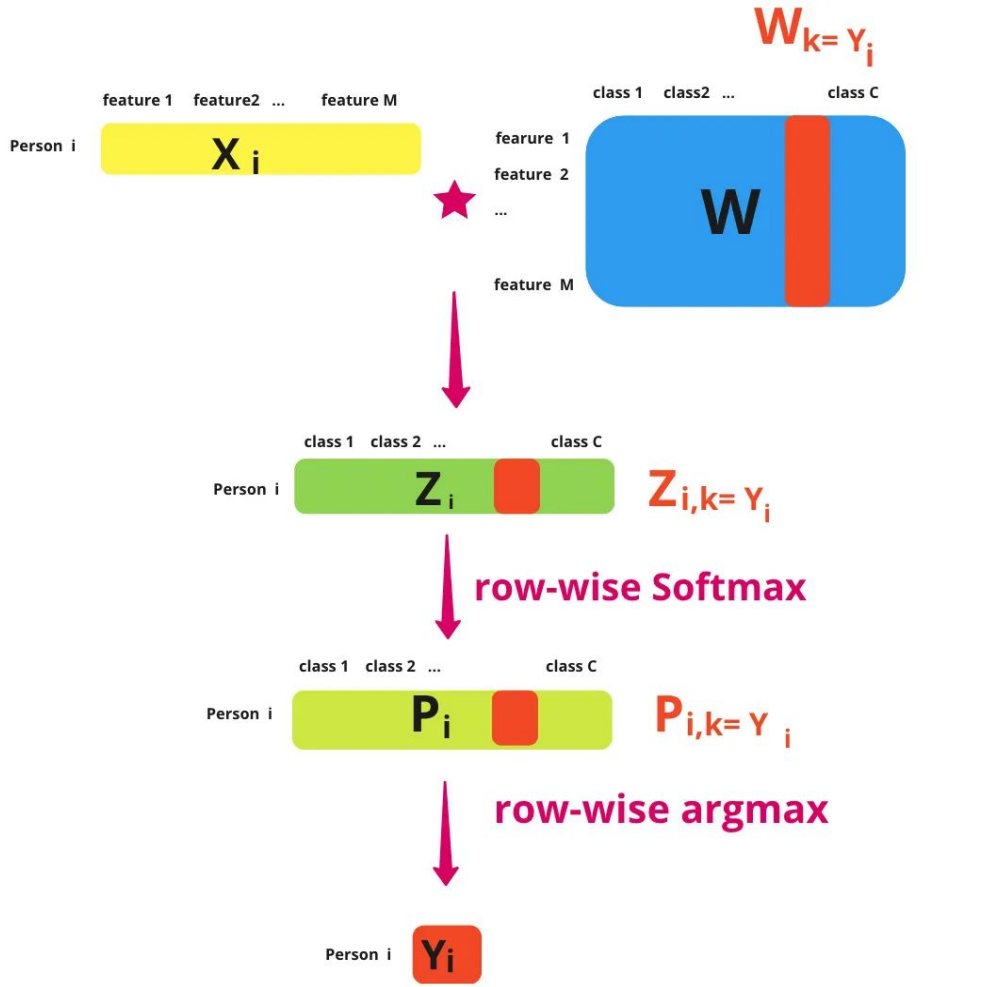
3 Prediction using Argmax

$$\hat{Y}_i = \text{argmax}(P_i)$$

Select class with highest probability per recipe



Likelihood



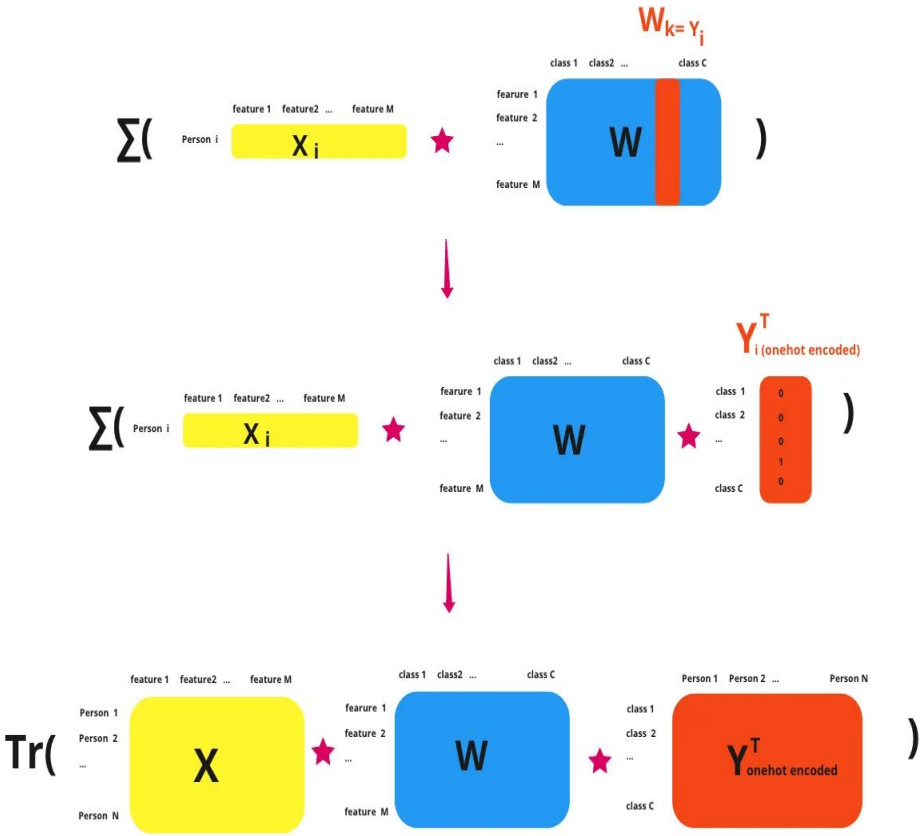
- $p(Y_i|X_i, W) = P_{i,k=Y_i} = \text{softmax}(Z_{i,k=Y_i}) = \frac{exp(Z_{i,k=Y_i})}{\sum_{k=0}^{C} exp(Z_{ik})} = \frac{\exp(-X_i W_{k=Y_i})}{\sum_{k=0}^{C} \exp(-X_i W_k)}$
- $p(Y|X, W) = \prod_{i=1}^{N} \frac{\exp(-X_i W_{k=Y_i})}{\sum_{k=0}^{C} \exp(-X_i W_k)}$

- We start from **Softmax probabilities**
- Extract the probability of the **true class**
- Multiply probabilities across all samples → **Entire dataset likelihood**
- Likelihood is used to decide how well the model fits → drives **weight updates**

Loss Function

$$W_{k=Y_i} = WY^T_{i(onehot_encoded)} \text{ and } \sum_{i=1}^{N} X_i W_{k=Y_i} = Tr(XWY^T_{onehot_encoded})$$

- We use **Negative Log-Likelihood (NLL)** as loss
- Measures how well predicted probabilities match true classes
- Loss is **averaged** (normalized) over all training samples
- Matrix form uses **trace (Tr)** operation → sum of diagonal elements
- Loss depends on predicted **softmax probabilities**



Regularizer and Gradient

$f(W)$

$=$ loss + regularization

$$= \frac{1}{N} \sum_{i=1}^{N} (X_i W_{k=Y_i} + \log \sum_{k=0}^{C} \exp(-X_i W_k)) + \mu ||W||^2$$

Gradient:

The gradient calculation is as follows. One thing to note that the gradient of $W_{k=Y_i}$

$\nabla_{W_k} f(W)$

$$= \frac{1}{N} \sum_{i=1}^{N} (X_i^T I_{[Y_i=k]} - X_i^T \frac{\exp(-X_i W_k)}{\sum_{k=0}^{C} \exp(-X_i W_k)}) + 2\mu W$$

$$= \frac{1}{N} \sum_{i=1}^{N} (X_i^T I_{[Y_i=k]} - X_i^T P_i) + 2\mu W$$

$$= \frac{1}{N} (\sum_{i=1}^{N} X_i^T I_{[Y_i=k]} - \sum_{i=1}^{N} X_i^T P_i) + 2\mu W$$

$$= \frac{1}{N} (X^T Y_{onehot_encoded} - X^T P) + 2\mu W$$

$$= \frac{1}{N} (X^T (Y_{onehot_encoded} - P)) + 2\mu W$$

- We add **L2 regularization** term to prevent overfitting
- Helps keep weights small and stable
- Combined function for optimization:

  f(W)=Loss+μ // W // 2f(W)=Loss+μ // W // 2
- μ (lambda) controls regularization strength
- Once loss and gradient are mathematically defined, we **optimize weights W** iteratively
- Initialize W=0 and update step-by-step using gradient information
- Update rule: **W:=W−η ∇f(W)**, where η= learning rate
- Regularization term included in gradient to prevent overfitting
- Continue updating until:
  - Loss **converges**
  - Or **max iterations** reached
- Our implementation uses **batch gradient descent** → uses entire dataset per update
- Slower but stable and ideal for demonstration

Results

Validation Accuracy: 0.5703298179619802

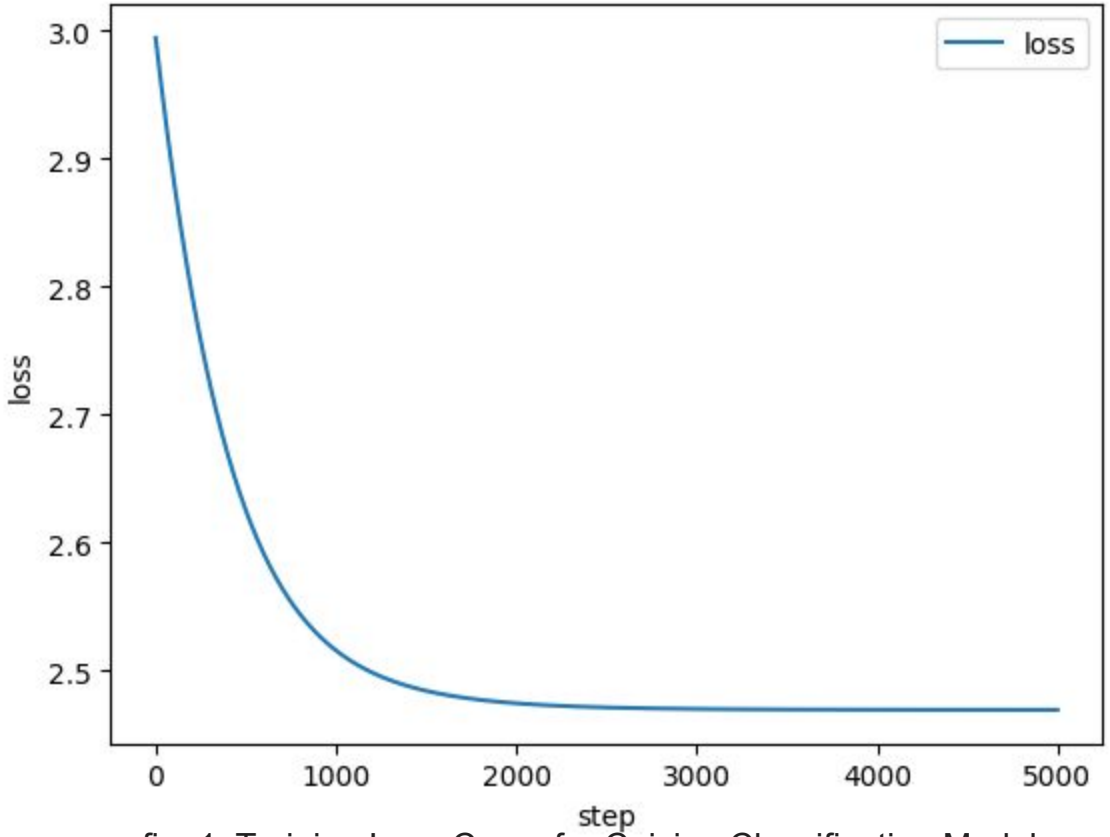
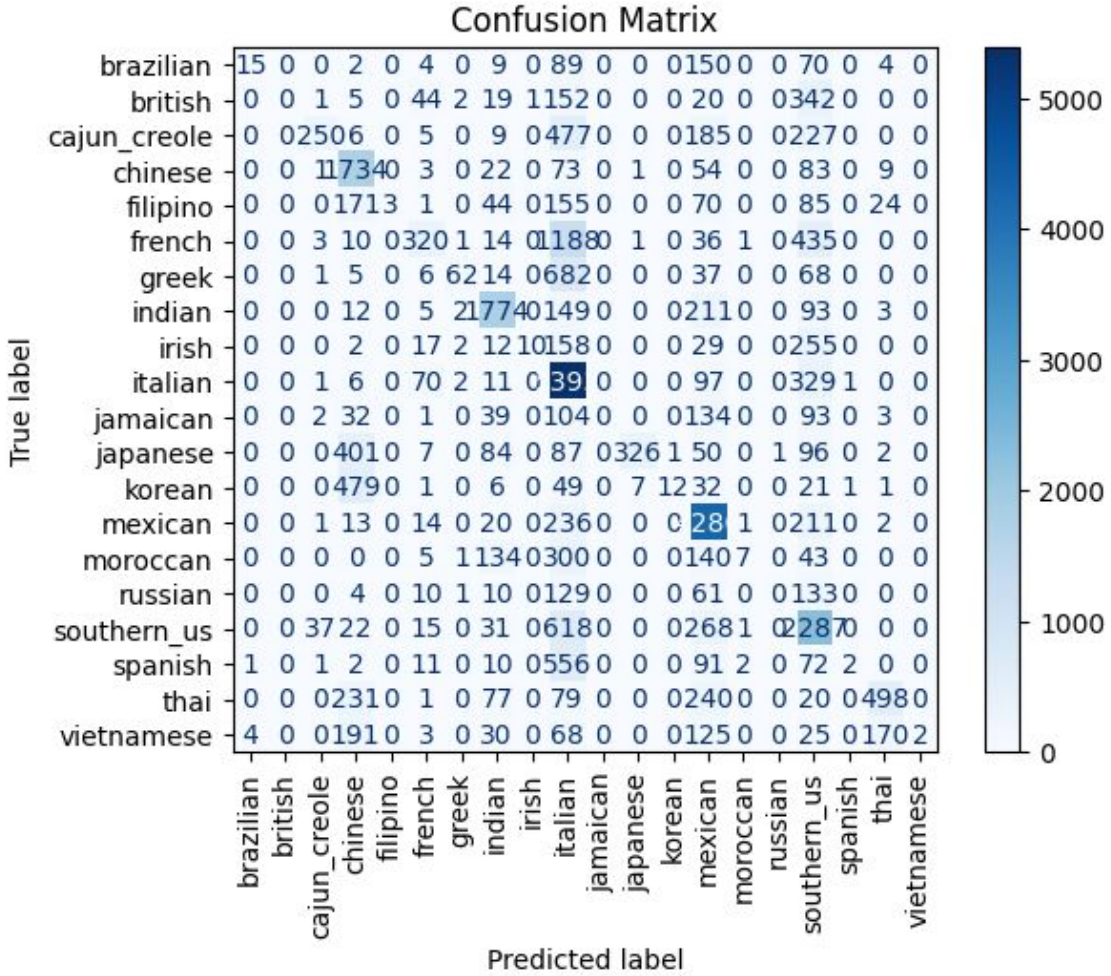


fig. 1: Training Loss Curve for Cuisine Classification Model



Conclusion

1. Successfully built a **multiclass cuisine classification model** from scratch
2. Extracted features using **TF-IDF ingredient representation**, enabling meaningful comparison across recipes
3. Implemented **softmax logistic regression manually**, including:
    a. Loss computation using **negative log-likelihood**
    b. **L2 regularization** to prevent overfitting
    c. Gradient descent for weight updates
4. Model demonstrated the ability to learn cuisine patterns from ingredient lists
5. Visual evaluation (loss curve) confirmed **stable convergence behavior**
6. Successfully generated predictions for **10,000 unseen recipes** in required submission format

Thank You!